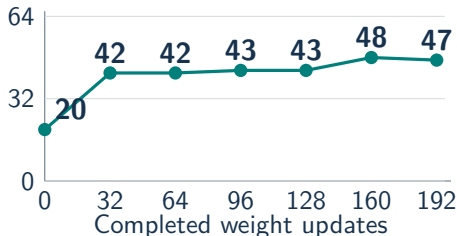


A small reproduction gives us a learning signal.

Goal: Train a recursive language model (RLM) to use code and delegate useful subtasks. First, learn reinforcement learning (RL) on checkable math answers.

Correct final answers / same 64 MATH500 questions



Authors' **Dr. GRPO** code; pretrained **Qwen2.5-Math-1.5B**; one A100.

Eight attempts per training question; reward correct final answers. No added supervised fine-tuning (SFT).

20 to 47 correct: improvement persists, with gains and regressions along the way.

Fixed 64-question progress subset, excluded from weight training; one greedy answer, same chat prompt, full math checker, 3,000-token cap. Not the paper's full benchmark result; final tests pending. Liu et al., *Understanding R1-Zero-Like Training* (2025).

Useful practice is neither all wrong nor all right.

Example: Solve $3x + 2 = 14$. Answer $x = 4$ earns 1; $x = 5$ earns 0.

Dr. GRPO compares rewards with their **same-question average**.

Eight attempts on one problem	Scores	Relative signal
All wrong	Eight zeros	None
Some right, some wrong	A mix of 1 and 0	Learn from the contrast
All right	Eight ones	None

Why curriculum could help

Practice problems the model can sometimes solve. As it improves, move toward harder problems that still produce useful contrast.

Contrast must occur within a question, not just across easy and hard questions. This concerns binary group-relative rewards, not all RL. Mixed scores do not guarantee learning. Related: DAPO's filtering of uniform-reward groups.

Existing skill needs a usable interface.

Our prompt check: same two models, same 500 MATH500 questions.

How we ask	Before RL	After short chat-style RL
Chat instructions	154 / 500	308 / 500
Question alone	305 / 500	317 / 500

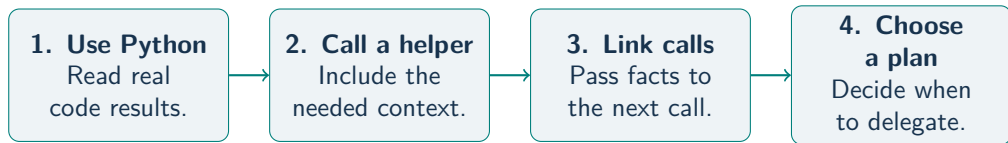
Lesson: A poor interface can hide ability and inflate the apparent training gain. Test a capable starting setup.

RLM implication: Before expecting good decomposition, show how to run Python, call a helper, pass the needed context, and read its real return.

SFT means learning by copying demonstrations; it is optional before RL. Our earlier RLM work already learned some executable routines, with limited transfer. The open question is what preparation enables flexible combinations, not whether we ever taught tool use.

Teach the interface, then practice useful choices.

Proposed curriculum: build on skills the model can already demonstrate.



Invented example: Document 1 says “Mira works at Aster Lab.” Document 2 says “Aster Lab is in Oxford.” To find Mira’s work city, pass *Aster Lab* to the next lookup, then use its returned city: *Oxford*.

Use demonstrations or SFT where needed; then let RL practice alternatives. More attempts and modest temperature changes may reveal rare successes.

Hypothesis, not an established curriculum. Progressively remove help, but keep some earlier skills in practice. Verify actual tool returns; reward task success, not extra calls. Related: RLM training, DeepSeek-R1 cold start.

Prior work supports the ingredients, not our sequence.

DAPO: spend updates on informative attempts

Its dynamic sampling removes groups that are all correct or all wrong. This directly connects to our reward-contrast lesson.

Recursive Language Models: The authors train a smaller model on filtered demonstrations of environment use and recursive calls.

DeepSeek-R1: Uses supervised preparation before RL; R1-Zero does not. Preparation is a design choice, not a universal requirement.

Our next questions: Which skills, teaching examples, exploration choices, and feedback make RLM learning work?

Sources: DAPO §3.2; RLM v3 §3.2 and Appendix A; DeepSeek-R1 §2.3.1. These motivate an experiment; they do not establish our curriculum or its novelty.

A portfolio of small experiments, not one settled story.

First locate the failure; then change one thing.

Curriculum is one promising idea. Missing skills, poor exploration, and misleading feedback need different remedies.

Question	Small comparison
Missing tool skills?	Prompt examples vs. protocol vs. linked-call SFT
Does teaching order help?	Same examples: staged vs. shuffled
Too little exploration?	More attempts or varied plans, same total budget
Can it use feedback?	Successful-only vs. error-and-repair examples

For each: Check before/after RL; test new combinations, not just longer inputs.

Proposals, not results. Match compute and helper models. Track real execution, correct information passing, and final answers. Replicate promising effects; keep failures.